

NIST SP 800-90A

Материал из Википедии — свободной энциклопедии

NIST SP 800-90A — («SP» — сокращение от англ. *Special Publication*», «специальная публикация») — публикация Национального института стандартов и технологий (англ. *NIST*) с названием «Рекомендация для генерации случайных чисел с использованием детерминированных генераторов случайных битов» (англ. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*»). Публикация содержит описания трех предположительно криптографически стойких генераторов псевдослучайных чисел для использования в криптографии: Hash_DRBG (на основе хеш-функций), HMAC_DRBG (на основе хеш-кода аутентификации сообщений) и CTR_DRBG (на основе блочных шифров в режиме счетчика).

С 24 июня 2015 года текущей версией издания является Редакция 1 (англ. *Revision 1*). Более ранние версии включали четвёртый генератор, Dual_EC_DRBG (основанный на эллиптической криптографии). Позже сообщалось, что Dual_EC_DRBG, вероятно, содержит клептографический бэкдор, внедрённый Агентством национальной безопасности, в то время как другие три генератора случайных чисел принимаются как непротиворечивые и безопасные несколькими криптографами^{[1][2]}.

NIST SP 800-90A является общественным достоянием и находится в свободном доступе, так как представляет из себя исследование федерального правительства США.

Содержание

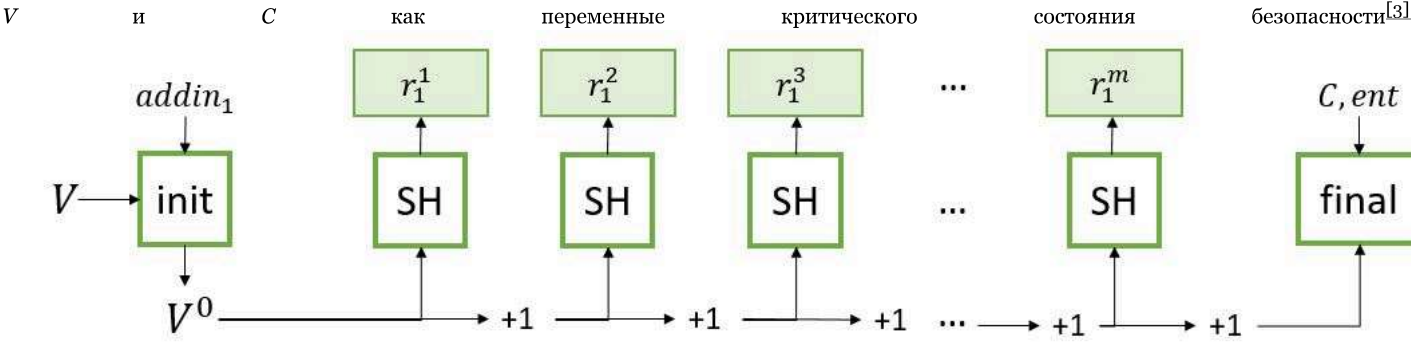
- Hash_DRBG
- HMAC_DRBG
- CTR_DRBG
- Бэкдор в Dual_EC_DRBG
- Анализ безопасности
- Примеры применения
- История версий NIST Special Publication 800-90A
- Примечания

Hash_DRBG

HASH-DRBG основан на хеш-функции SH : {0, 1}^{*} → {0, 1}^l из семейства криптографических хеш-функций SHA^[3]. Состояние имеет вид S = (V, C, cnt), где V ∈ {0, 1}^{len} — счетчик, который хешируется для создания конечных блоков, значение которого обновляется во время каждого вызова генератора; C — константа, зависящая от порождающего элемента (англ. *seed*), а cnt — счетчик повторного заполнения. cnt указывает количество запросов псевдослучайных битов с момента получения нового значения, принятого от истинно случайного генератора во время создания экземпляра или повторного заполнения.

Алгоритм генерации для HASH-DRBG. Если в вызове generate используется дополнительный вход, он хэшируется и добавляется в счетчик V по модулю 2^{len} во время процесса инициализации. Выходные блоки r_{*j*} затем формируются путем хеширования счетчика V. В конце вызова счетчик хэшируется с отдельным префиксом, а результирующая строка вместе с константой C и cnt добавляется к V, результат такой операции задается как обновленный счетчик.

Константа C обновляется только во время повторного заполнения (когда она снова является производной от новой переменной V) и добавляется в переменную состояния V во время каждого обновления состояния. Константа C позволяет убедиться, что даже если предыдущий счетчик V дублируется в какой-то момент в разных периодах повторного заполнения (почти наверняка различных), счетчик при следующем обновлении значения будет препятствовать повторению предыдущей последовательности состояний. Стандарт определяет



```

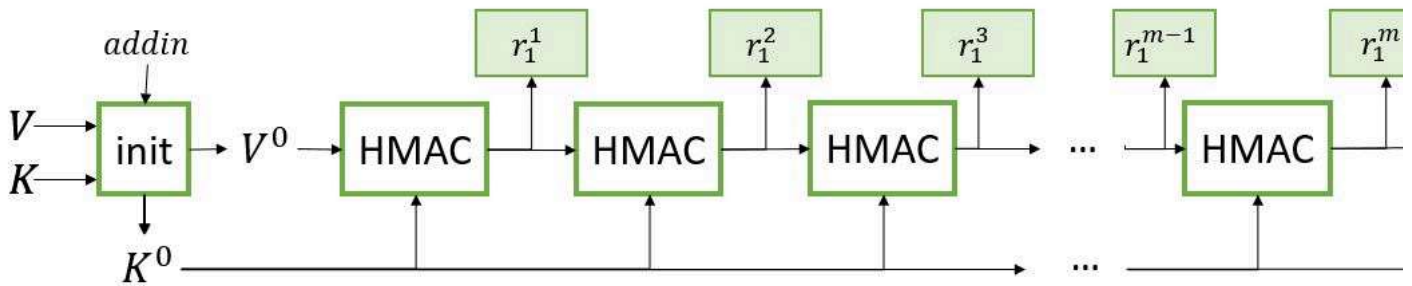
temp ← ε
m ← ⌊β/1⌋
for j = 1, . . . , m do
  r(j) ← SH(V(j-1))
  V(j) ← (V(j-1) + 1) mod 2len
  temp ← temp || r(j)
R ← left(temp, β)
H ← SH(0x03 || V(0))
V' ← (V(0) + H + C + cnt) mod 2len
cnt' ← cnt + 1
return (V', C, cnt')

```

HMAC_DRBG

HMAC — это код аутентификации (проверки подлинности) сообщений, использующий хеш-функции, который был введен Михиром Беллэром (англ. *Mihir Bellare*) и его коллегами^[4] и впоследствии стандартизирован^[5]. HMAC-DRBG использует HMAC: $\{0, 1\}^l \times \{0, 1\}^* \rightarrow \{0, 1\}^l$ для генерации блоков псевдослучайного вывода. Состояние имеет вид $S = (K, V, cnt)$, где стандарт определяет K и V как критические для безопасности переменные секретного состояния. Предполагается, что после инициализации начальным состоянием является $S_0 = (K_0, V_0, cnt_0)$, где $cnt_0 = 1$ и $K_0, V_0 \leftarrow \{0, 1\}^{len}$. Здесь $K \in \{0, 1\}^l$ используется в качестве ключа HMAC, $V \in \{0, 1\}^l$ является счетчиком, и cnt обозначает счетчик повторного заполнения^[3].

Алгоритм генерации использует функцию `update`, которая используется для добавления любых дополнительных входных данных в переменные состояния K и V , и обновляет оба в одностороннем порядке. Если в вызов включены дополнительные входные данные, выполняется дополнительная пара обновлений. Сам алгоритм генерации для HMAC-DRBG работает следующим образом: процесс инициализации добавляет любые дополнительные входные данные в переменные состояния через функцию `update`; если дополнительные входные данные не включены в вызов, состояние остается неизменным. За K_0, V_0 обозначим переменные состояния, соответствующие этому процессу, после чего результирующие блоки автоматически генерируются путем итеративного применения алгоритма $HMAC(K_0, \cdot)$ для текущего счетчика V_{j-1} , выходному блоку r_j и следующему значению счетчика V_j приравнивается результирующая строка. Ключ K_0 остается неизменным на протяжении каждой итерации. По завершении вызова окончательный процесс обновляет K и V с помощью функции `update`^[3].



Функция `update`

```

Входные данные: addin, K, V
Выходные данные: K, V
K ← HMAC(K, V || 0x00 || addin)
V ← HMAC(K, V)
if addin ≠ ε then
  K ← HMAC(K, V || 0x01 || addin)
  V ← HMAC(K, V)
return (K, V)

```

функция `generate`

```

Входные данные: S = (K, V, cnt), β, addin
Выходные данные: S' = (K', V', cnt'), R
if 0 ← check(S, β, addin) then
  return (error, ⊥)
if addin ≠ ε then
  (K0, V0) ← update(addin, K, V)
else (K0, V0) ← (K, V)
temp ← ε; m ← ⌊β/1⌋
for j = 1, . . . , m do
  V(j) ← HMAC(K0, V(j-1))
  r(j) ← V(j)
  temp ← temp || r(j)
R ← left(temp, β)
(K', V') ← update(addin, K0, V(m))
cnt' ← cnt + 1
return (R, (K', V', cnt'))

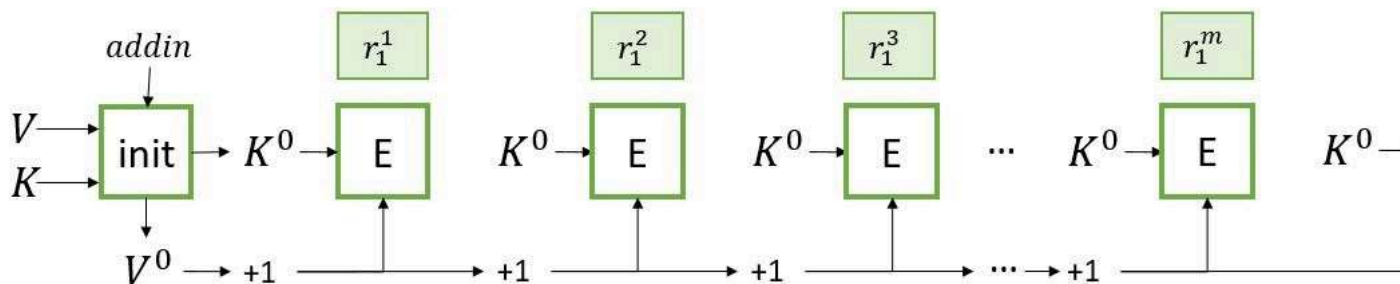
```

CTR_DRBG

CTR_DRBG основан на алгоритме блочного шифра, функция шифрования которого $E : \{0, 1\}^k \times \{0, 1\}^l \rightarrow \{0, 1\}^l$ используется в CTR-режиме (режиме счетчика). Состояние имеет вид $S = (K, V, cnt)$, где $K \in \{0, 1\}^k$ используется в качестве ключа для блочного шифра, $V \in \{0, 1\}^l$ является счетчиком, а cnt обозначает счетчик повторного заполнения. Стандарт утверждает, что K и V являются критическими переменными состояния безопасности. Инициализация возвращает начальное состояние $S_0 = (K_0, V_0, cnt_0)$, где $cnt_0 = 1$ и $K_0 \leftarrow \{0, 1\}^k, V_0 \leftarrow \{0, 1\}^l$ ^[3].

Как и в HMAC_DRBG алгоритм использует функцию update для одностороннего обновления переменных состояния K и V , а также для включения любых дополнительных входов данных (которые передаются функции update с помощью параметра provided_data). Функция запускает блочный шифр в CTR-режиме с использованием текущего ключа и счетчика, объединяя результирующие выходные блоки r^j . Затем крайние слева $\kappa + l$ бит отсекаются, и значения provided_data встраиваются внутрь с помощью операции XOR. Функция update вызывается каждым из других процессов таким образом, что это гарантирует, что provided_data точно имеет длину в $\kappa + l$ бит^[3].

Существует два варианта CTR_DRBG в зависимости от того, используется ли производящая функция. Производящая функция CTR_DRBG df объединяет в себе функцию, основанную на CBC-MAC, и возможность извлечения почти равномерного выхода из достаточно высокоэнтропийных входов. В случае, если производящая функция df не используется, строки дополнительного входа $addin$ ограничены до не более $(\kappa + l)$ — бит в длину. Процесс инициализации подключает каждый дополнительный вход к функции update: если производящая функция используется, то строка дополнительного входа представляет из себя строку $(\kappa + l)$ -бит с CTR_DRBG df до начала этого процесса. Если дополнительный вход не используется, состояние остается неизменным, и $addin$ имеет значение $0^{(\kappa+l)}$ (для формирования входных данных для функции update во время финальной процедуры при завершении вызова). Выходные блоки r^j затем создаются итеративно с помощью блочного шифра в CTR-режиме. Ключ K остаётся неизменным на протяжении всех этих итераций. При завершении вызова окончательный процесс обновляет оба K и V через вызов функции update^[3].



Функция update

```

Входные данные: provided data, K, V
Выходные данные: K, V
temp ← ε
m ← [(κ + l)/l]
for j = 1, . . . , m do
  V ← (V + 1) mod 2^l
  C(i) ← E(K, V)
  temp ← temp || C(i)
temp ← left(temp, (κ + l))
K || V ← temp ⊕ provided data
return (K, V)

```

функция generate

```

Входные данные: S = (K, V, cnt), β, addin
Выходные данные: S' = (K', V', cnt'), R
if 0 ← check(S, β, addin) then
  return (error, 1)
if addin ≠ ε then
  if derivation function used then
    addin ← df(addin, (κ + l))
  else if len(addin) < (κ + l) then
    addin ← addin || 0^(κ+l-len(addin))
  (K0, V0) ← update(addin, K, V)
else
  addin ← 0^(κ+l); (K0, V0) ← (K, V)
temp ← ε; m ← [β/l]
for j = 1, . . . , m do
  V(j) ← (V(j-1) + 1) mod 2^l
  r(j) ← E(K0, V(j))
  temp ← temp || r(j)
R ← left(temp, β)
(K', V') ← update(addin, K0, V(m))
cnt' ← cnt + 1
return (R, (K', V', cnt'))

```

Бэкдор в Dual_EC_DRBG

Dual_EC_DRBG была изъята из публикации с выпуском первой редакции документа. Причиной этому стало потенциальное существование бэкдора. Бэкдор — намеренно встроенный дефект алгоритма, позволяющий получить несанкционированный доступ к данным или удалённому управлению компьютером^[6].

В рамках программы Bullrun АНБ вставляет бэкдоры в криптографические системы. Dual_EC_DRBG был предположительно одной из целей в 2013 году^[7]. В процессе работ по стандартизации, АНБ добилось того, что в конечном итоге стало единственным редактором стандарта^[8]. При получении доступа к Dual_EC_DRBG, принятого в NIST SP 800-90A, АНБ процитировало использование Dual_EC_DRBG известной охранной фирмой RSA Security в своих продуктах. Однако компании RSA Security было выплачено NSA в размере 10 миллионов долларов США за использование Dual_EC_DRBG по умолчанию в своих продуктах. Reuters описывает эту сделку как «обработанную бизнес-лидерами, а не чистыми технологами». Поскольку контракт на 10 миллионов долларов, чтобы заставить RSA Security использовать Dual_EC_DRBG, был описан Reuters как секретный, люди, участвовавшие в процессе принятия Dual_EC_DRBG в NIST SP 800-90A, предположительно не были осведомлены об этом очевидном конфликте интересов^[9]. Это может помочь объяснить, как генератор случайных чисел, который позже показал, что он уступает альтернативным аналогам (в дополнение к вероятному бэкдору), приняли как стандарт в NIST SP 800-90A.

Хотя потенциал для бэкдора в Dual_EC_DRBG уже был задокументирован Дэном Шумовым и Нильсом Фергюсоном в 2007 году,^[10] он продолжал использоваться в выпускаемых продуктах такими компаниями, как RSA Security до 2013 года^[1]. Учитывая известные недостатки в Dual_EC_DRBG, впоследствии появились обвинения в том, что RSA Security сознательно вставила бэкдор NSA в свои продукты. RSA отрицает сознательную вставку бэкдора в свои продукты^[11].

После того, как бэкдор был обнаружен, NIST возобновил процесс государственной проверки стандарта NIST SP 800-90A^{[7][12]}. Пересмотренная версия NIST SP 800-90A, в которой Dual_EC_DRBG был удален, опубликована в июне 2015 года^[13].

Анализ безопасности

Hash_DRBG и HMAC_DRBG имеют доказательства безопасности для генерации псевдослучайных последовательностей^[14]. Документ, подтверждающий безопасность Hash_DRBG и HMAC_DRBG цитирует попытки доказательства безопасности для Dual_EC_DRBG, высказывая, что не следует использовать CTR_DRBG, потому что это единственный генератор в NIST SP 800-90A, для которого отсутствуют доказательства безопасности^[14].

HMAC_DRBG также имеет машинно-подтвержденное доказательство безопасности^[15]. Тезис, содержащий проверенное вычислительными методами доказательство безопасности, также доказывает, что взлом правильно реализованного экземпляра HMAC_DRBG не ставит под угрозу безопасность чисел, созданных до взлома^[15].

Было показано, что CTR_DRBG имеет проблемы с безопасностью при использовании с определёнными параметрами, поскольку криптографы не учитывали размер блока шифра при проектировании этого генератора псевдослучайных чисел^[16]. CTR_DRBG кажется безопасным и неотличимым от истинного случайного источника, когда AES используется в качестве базового блочного шифра и 112 бит берутся из этого генератора псевдослучайных чисел^[16]. Когда AES используется в качестве базового блочного шифра и 128 бит берутся из каждого экземпляра, необходимый уровень безопасности ставится с оговоркой, что на выходе 128-битный шифр в режиме счетчика можно отличить от истинного генератора случайных чисел^[16]. Когда AES используется в качестве базового блочного шифра и более 128 бит берется из этого генератора псевдослучайных чисел, тогда результирующий уровень безопасности ограничивается размером блока, а не размером ключа, и поэтому фактический уровень безопасности намного меньше, чем уровень безопасности, подразумеваемый размером ключа^[16]. Также показано, что CTR_DRBG не может обеспечить ожидаемый уровень безопасности при использовании Triple DES, поскольку его 64-битный размер блока намного меньше, чем 112-битный размер ключа, используемый для Triple DES^[16].

Попытка доказательства безопасности для Dual_EC_DRBG утверждает, что для обеспечения безопасности Dual_EC_DRBG требуется, чтобы три задачи принадлежали классу NP: Задача Диффи-Хеллмана, проблема дискретного логарифмирования и проблема усеченной точки^[17]. Проблема принятия решений Диффи-Хеллмана широко признана как математически трудная^[17]. Проблема дискретного логарифмирования широко не принята к классу NP, некоторые доказательства показывают, что эта проблема трудна, но не доказывают этого^[17]. Доказательство безопасности, следовательно, подлежит сомнению и может быть принято недействительным, если проблема дискретного логарифмирования будет показана разрешимой, а не математически трудной. Проблема усеченной точки требует, чтобы достаточное количество битов было усечено от точки, выбранной Dual_EC_DRBG, чтобы сделать её неотличимой от действительно случайного числа^[17], однако было показано, что усечение 16 бит, заданное по умолчанию стандартом Dual_EC_DRBG, является недостаточным для того, чтобы сделать вывод неотличимым от истинного генератора случайных чисел^[18] и поэтому делает недействительным доказательство безопасности Dual_EC_DRBG при использовании значения усечения по умолчанию.

Примеры применения

Приведенные алгоритмы являются стандартами и используются крупными компаниями для создания собственных продуктов на их основе. Так компания Microsoft в процессе создания обновления для своего CryptoApi под названием «Cryptography API: Next Generation (CNG)» установила в качестве генератора псевдослучайных чисел по умолчанию на CTR_DRBG^[19].

Компания Intel в инструкции RdRand для генерации случайного числа при помощи встроенного генератора случайных чисел также использует CTR_DRBG^[20].

История версий NIST Special Publication 800-90A

- Kelsey, John; Barker, Elaine*. NIST Special Publication 800-90: Recommendation for Random Number Generation Using Deterministic Random Bit Generators (<http://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-90.pdf>) (PDF). National Institute of Standards and Technology (июнь 2006). Дата обращения: 27 ноября 2016. Архивировано (<https://web.archive.org/web/20161128133953/http://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-90.pdf>) 28 ноября 2016 года. Отозвана в марте 2007.
- Kelsey, John; Barker, Elaine*. NIST Special Publication 800-90: Recommendation for Random Number Generation Using Deterministic Random Bit Generators (Revised) (<http://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-90r.pdf>) (PDF). National Institute of Standards and Technology (март 2007). Дата обращения: 27 ноября 2016. Архивировано (<https://web.archive.org/web/20161128195241/http://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-90r.pdf>) 28 ноября 2016 года. Отозвана в январе 2012.
- Kelsey, John; Barker, Elaine*. NIST Special Publication 800-90A: Recommendation for Random Number Generation Using Deterministic Random Bit Generators (<http://csrc.nist.gov/publications/nistpubs/800-90A/SP800-90A.pdf>) (PDF). National Institute of Standards and Technology (январь 2012). Дата обращения: 19 ноября 2016. Архивировано (<https://web.archive.org/web/20131009210654/http://csrc.nist.gov/publication/s/nistpubs/800-90A/SP800-90A.pdf>) 9 октября 2013 года. Отозвана в июне 2015.
- Kelsey, John; Barker, Elaine*. NIST Released Special Publication (SP) 800-90A Revision 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators (<http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-90Ar1.pdf>) (PDF). National Institute of Standards and Technology (июнь 2015). doi:10.6028/NIST.SP.800-90Ar1 (<https://dx.doi.org/10.6028/NIST.SP.800-90Ar1>). Дата обращения: 19 ноября 2016. Архивировано (<https://web.archive.org/web/20161201045644/http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-90Ar1.pdf>) 1 декабря 2016 года.

Примечания

1. *Green, Matthew*. RSA warns developers not to use RSA products (<http://blog.cryptographyengineering.com/2013/09/rsa-warns-developers-against-its-own.html>) (20 сентября 2013). Дата обращения: 23 августа 2014. Архивировано (<https://web.archive.org/web/20131010085457/http://blog.cryptographyengineering.com/2013/09/rsa-warns-developers-against-its-own.html>) 10 октября 2013 года.
2. *Schneier, Bruce*. The Strange Story of Dual_EC_DRBG (https://www.schneier.com/blog/archives/2007/11/the_strange_sto.html) (15 ноября 2007). Дата обращения: 25 ноября 2016. Архивировано (https://web.archive.org/web/20190423212823/https://www.schneier.com/blog/archives/2007/11/the_strange_sto.html) 23 апреля 2019 года.
3. *Woodage, Joanne; Shumow, Dan*. An Analysis of the NIST SP 800-90A Standard (<https://eprint.iacr.org/2018/349.pdf>). National Institute of Standards and Technology (апрель 2018). Дата обращения: 25 ноября 2016. Архивировано (<https://web.archive.org/web/20190218001505/https://eprint.iacr.org/2018/349.pdf>) 18 февраля 2019 года.
4. *Bellare, Mihir; Canetti, Ran; Krawczyk, Hugo*. Keying hash functions for message authentication (англ.). — *Crypto*, Volume 96, pages 1-15, 1996.
5. *Turner, James*. The keyed-hash message authentication code (hmac) (англ.). — *Federal Information Processing Standards*, 2008.
6. J.P. Aumasson Cryptographic backdooring (http://cryptome.org/2014/11/backdooring_nsc14.pdf) Архивная копия (https://web.archive.org/web/20191221162738/http://cryptome.org/2014/11/backdooring_nsc14.pdf) от 21 декабря 2019 на Wayback Machine
7. *Perloth, Nicole*. Government Announces Steps to Restore Confidence on Encryption Standards (<https://bits.blogs.nytimes.com/2013/09/10/government-announces-steps-to-restore-confidence-on-encryption-standards/>). *New York Times* (10 сентября 2013). Дата обращения: 23 августа 2014. Архивировано (<https://web.archive.org/web/20140712084931/http://bits.blogs.nytimes.com/2013/09/10/government-announces-steps-to-restore-confidence-on-encryption-standards/>) 12 июля 2014 года.
8. *Greenwald, Glenn; Ball, James; Berger, Julian*. Revealed: how US and UK spy agencies defeat internet privacy and security (<https://www.theguardian.com/world/2013/sep/05/nsa-gchq-encryption-codes-security>). *The Guardian* (5 сентября 2013). Дата обращения: 23 августа 2014. Архивировано (<https://web.archive.org/web/20130918135152/http://www.theguardian.com/world/2013/sep/05/nsa-gchq-encryption-codes-security>) 18 сентября 2013 года.
9. Menn, Joseph (20 декабря 2013). Exclusive: Secret contract tied NSA and security industry pioneer (<https://www.reuters.com/article/2013/12/20/us-usa-security-rsa-idUSBRE9BJ1C220131220>). *Reuters*. Архивировано (<https://web.archive.org/web/20150924191918/http://www.reuters.com/article/2013/12/20/us-usa-security-rsa-idUSBRE9BJ1C220131220>) 24 сентября 2015. Дата обращения: 23 августа 2014.
10. Bruce Schneier (15 ноября 2007). Did NSA Put a Secret Backdoor in New Encryption Standard? (<https://www.wired.com/2007/11/securitymatters-1115/>). *Wired News*. Архивировано (<https://web.archive.org/web/20151123065121/http://www.wired.com/2007/11/securitymatters-1115/>) 23 ноября 2015. Дата обращения: 23 августа 2014.
11. *Goodin, Dan*. We don't enable backdoors in our crypto products, RSA tells customers (<https://arstechnica.com/security/2013/09/we-dont-enable-backdoors-in-our-crypto-products-rsa-tells-customers/>). *Ars Technica* (20 сентября 2013). Дата обращения: 23 августа 2014. Архивировано (<https://web.archive.org/web/20141012130722/http://arstechnica.com/security/2013/09/we-dont-enable-backdoors-in-our-crypto-products-rsa-tells-customers/>) 12 октября 2014 года.
12. NIST Invites Comments on Draft SP 800-90A, Revision 1 (<http://csrc.nist.gov/groups/ST/toolkit/800-90A-RFC.html>). National Institute of Standards and Technology (21 апреля 2014). Дата обращения: 23 августа 2014. Архивировано (<https://web.archive.org/web/20140723001417/http://csrc.nist.gov/groups/ST/toolkit/800-90A-RFC.html>) 23 июля 2014 года.
13. *Kelsey, John; Barker, Elaine*. NIST Released Special Publication (SP) 800-90A Revision 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators (<http://csrc.nist.gov/publications/nistpubs/800-90A/SP800-90A.pdf>) (PDF). National Institute of Standards and Technology (июнь 2015). doi:10.6028/NIST.SP.800-90Ar1 (<https://dx.doi.org/10.6028/NIST.SP.800-90Ar1>). Дата обращения: 19 ноября 2016. Архивировано (<http://web.archive.org/web/20131009210654/http://csrc.nist.gov/publications/nistpubs/800-90A/SP800-90A.pdf>) 9 октября 2013 года.
14. *Kan, Wilson*. Analysis of Underlying Assumptions in NIST DRBGs (<http://eprint.iacr.org/2007/345.pdf>) (PDF) (4 сентября 2007). Дата обращения: 19 ноября 2016. Архивировано (<https://web.archive.org/web/20170202002318/https://eprint.iacr.org/2007/345.pdf>) 2 февраля 2017 года.
15. *Ye, Katherine Qinru*. The Notorious PRG: Formal verification of the HMAC-DRBG pseudorandom number generator (<https://www.cs.cmu.edu/~kqy/resources/thesis.pdf>) (PDF) (апрель 2016). Дата обращения: 19 ноября 2016. Архивировано (<https://web.archive.org/web/20161120150249/https://www.cs.cmu.edu/~kqy/resources/thesis.pdf>) 20 ноября 2016 года.
16. *Campagna, Matthew J*. Security Bounds for the NIST Codebook-based Deterministic Random Bit Generator (<http://eprint.iacr.org/2006/379.pdf>) (PDF) (1 ноября 2006). Дата обращения: 19 ноября 2016. Архивировано (<https://web.archive.org/web/20170202002237/http://eprint.iacr.org/2006/379.pdf>) 2 февраля 2017 года.
17. *Brown, Daniel R. L.; Gjosteen, Kristian*. A Security Analysis of the NIST SP 800-90 Elliptic Curve Random Number Generator (<http://eprint.iacr.org/2007/048.pdf>) (PDF) (15 февраля 2007). Дата обращения: 19 ноября 2016. Архивировано (<https://web.archive.org/web/20160328015857/http://eprint.iacr.org/2007/048.pdf>) 28 марта 2016 года.
18. *Schoenmakers, Berry; Sidorenko, Andrey*. Cryptanalysis of the Dual Elliptic Curve Pseudorandom Generator (<http://eprint.iacr.org/2006/190.pdf>) (PDF) (29 мая 2006). Дата обращения: 20 ноября 2016. Архивировано (<https://web.archive.org/web/20160308045321/http://eprint.iacr.org/2006/190.pdf>) 8 марта 2016 года.
19. FIPS PUB 186-2 (<http://csrc.nist.gov/publications/fips/archive/fips186-2/fips186-2.pdf>). *Federal Information Processing Standards*. National Institute of Standards and Technology (27 января 2000). Дата обращения: 13 января 2010. Архивировано (<https://web.archive.org/web/20110812232347/http://csrc.nist.gov/publications/fips/archive/fips186-2/fips186-2.pdf>) 12 августа 2011 года.
20. Intel Digital Random Number Generator (DRNG) Software Implementation Guide (<http://software.intel.com/en-us/articles/intel-digital-random-number-generator-drng-software-implementation-guide>) Архивная копия (<https://web.archive.org/web/20140112161155/http://software.intel.com/en-us/articles/intel-digital-random-number-generator-drng-software-implementation-guide>) от 12 января 2014 на Wayback Machine // Gael Hofemeier, 08/08/2012]

Источник — https://ru.wikipedia.org/w/index.php?title=NIST_SP_800-90A&oldid=146803598

Эта страница в последний раз была отредактирована 18 июля 2025 года в 08:58.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации «Фонд Викимедиа» (Wikimedia Foundation, Inc.)